

3 OSNOVNE KONTROLNE STRUKTURE

- 3.1 *Izvršne instrukcije viših programskih jezika*
- 3.2 *Standardni i strukturirani dijagrami toka*
- 3.3 *Temeljne kontrolne strukture*
 - 3.3.1 Sekvenca BEGIN-END
 - 3.3.2 Uvjetna grananja
 - 3.3.3 Petlje
 - 3.3.4 Skokovi
- 3.4 *Ulaz i izlaz podataka*
 - 3.4.1 Ulaz i izlaz podataka pomoću ulaznih i izlaznih uređaja računala
 - 3.4.2 Konverzija podataka tipa A
 - 3.4.3 Konverzija podataka tipa X
 - 3.4.4 Konverzija podataka tipa I
 - 3.4.5 Konverzija podataka tipa F
 - 3.4.6 Konverzija podataka tipa E
- 3.5 *Klasifikacija osnovnih kontrolnih struktura*
- 3.6 *Kontrolne strukture podržane od strane programskih jezika 3GL*

3 OSNOVNE KONTROLNE STRUKTURE

3.1 Izvršne instrukcije viših programskih jezika

- **instrukcije prijenosa i obrade podataka** (obavljaju se onim redoslijedom kojim su navedene u programu)
- **kontrolne instrukcije** (određuju kojim će se redoslijedom obavljati pojedine operacije prijenosa i obrade podataka)
- kontrolne instrukcije utječu na efikasnost, korektnost i razumljivost svakog programa

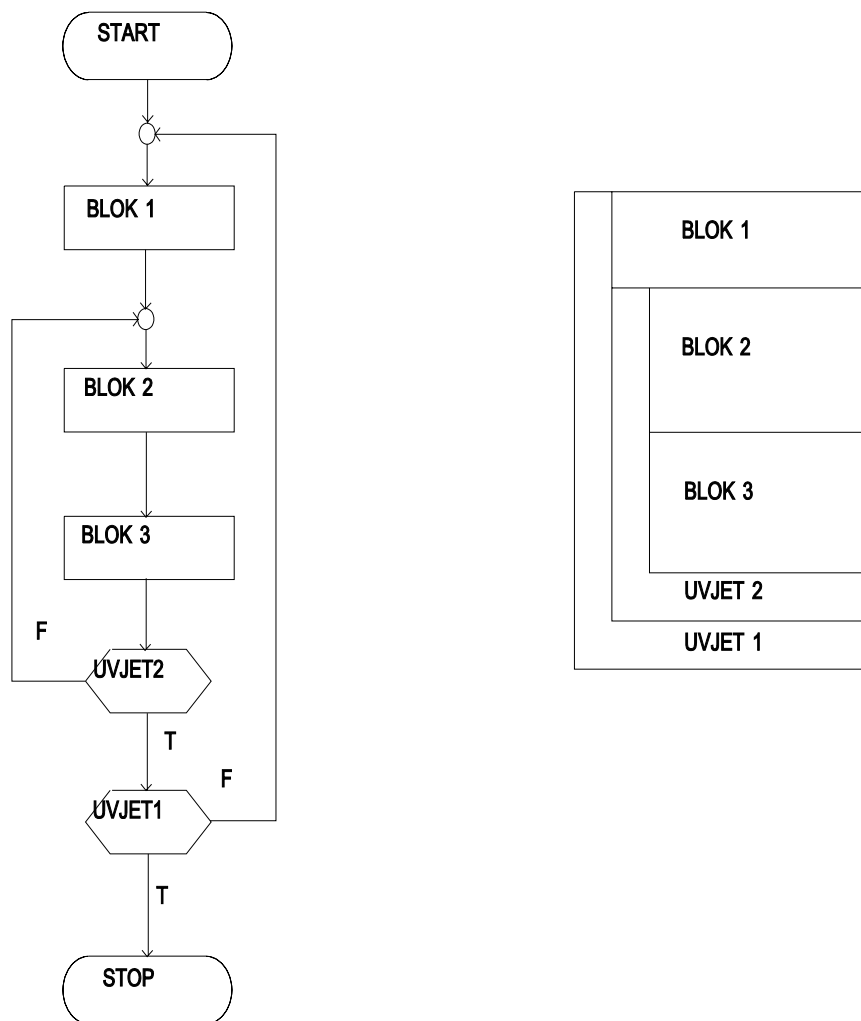
Veoma velika komponenta troškova obrade podataka su troškovi održavanja programske opreme koji su posljedica radova na ispravljanju grešaka koje se detektiraju tokom eksploatacije programske opreme, kao i radova na proširenju i modifikaciji programa.

Ako se striktno podržavaju principi strukturiranog programiranja i koristi konačan broj osnovnih kontrolnih struktura onda svaki programer koji čita programe nekog drugog programera u njima nalazi prepoznatljive standardne kontrolne strukture.

Kontrolne instrukcije se obično dijele u četiri grupe: (1) sekvence, (2) selekcije (grananja), (3) iteracije i (4) skokovi.

3.2 Standardni i strukturirani dijagrami toka

Primjer ekvivalentnih standardnih i strukturiranih (ili Nassi-Shneiderman-ovih) dijagrama toka



Slika 3.1 : Standardni i strukturirani dijagram toka

3.3 Temeljne kontrolne strukture

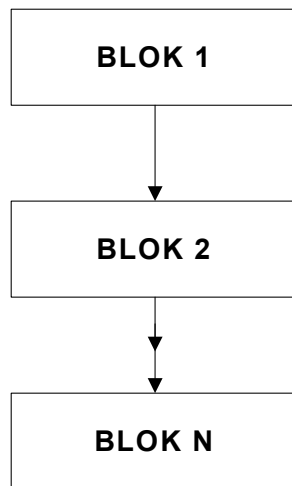
3.3.1 Sekvenca BEGIN-END

a)

```

BEGIN
    Blok 1 ;
    Blok 2 ;
    .....;
    Blok N
END
  
```

b)



3.3.2 Uvjetna grananja

3.3.2.1 Uvjet s dvije grane (IF-THEN-ELSE)

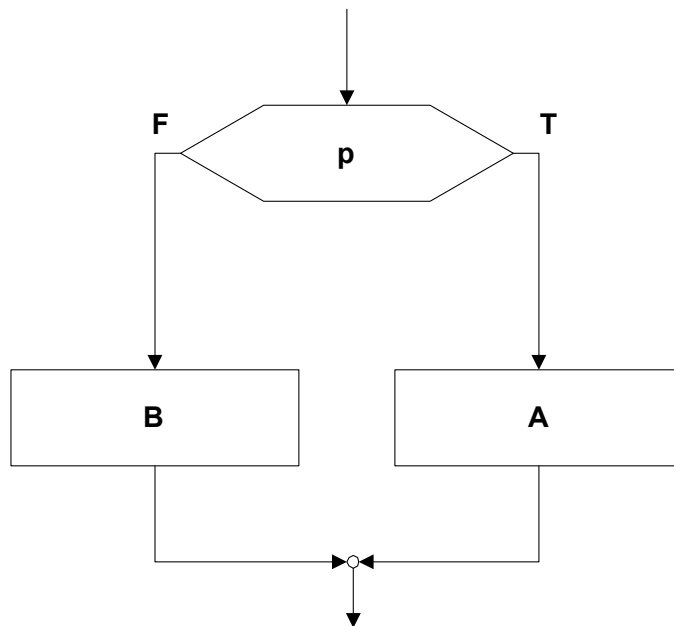
a)

IF p THEN A ELSE B END_IF

IF $x < y$ THEN $m := y$ END_IF

```
IF p
THEN
    BEGIN
        A1 ;
        ...;
        AN
    END
ELSE
    BEGIN
        B1 ;
        ...;
        BK
    END
END_IF
```

b)



3.3.2.2 Uvjet s jednom granom (IF-THEN)

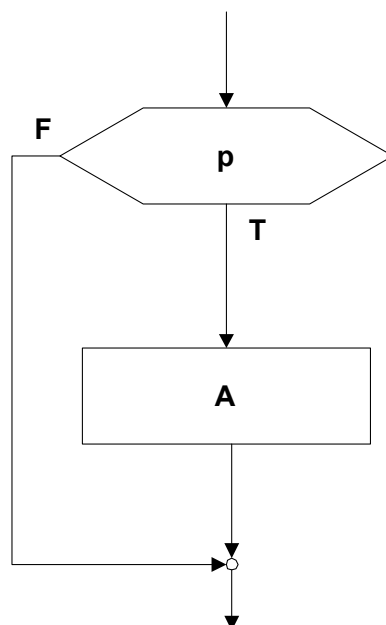
a)

IF p THEN A END_IF

a := x ;

IF x < 0 THEN a := -x END_IF

b)



3.3.2.3 Višegransko usmjeravanje (grananje tipa CASE)

Ako selektor uzme vrijednost a, b,..., z onda se respektivno izvršavaju blokovi A, B,..., Z. Ako je vrijednost selektora različita od a, b,..., z onda se izvršava specijalni blok Y.

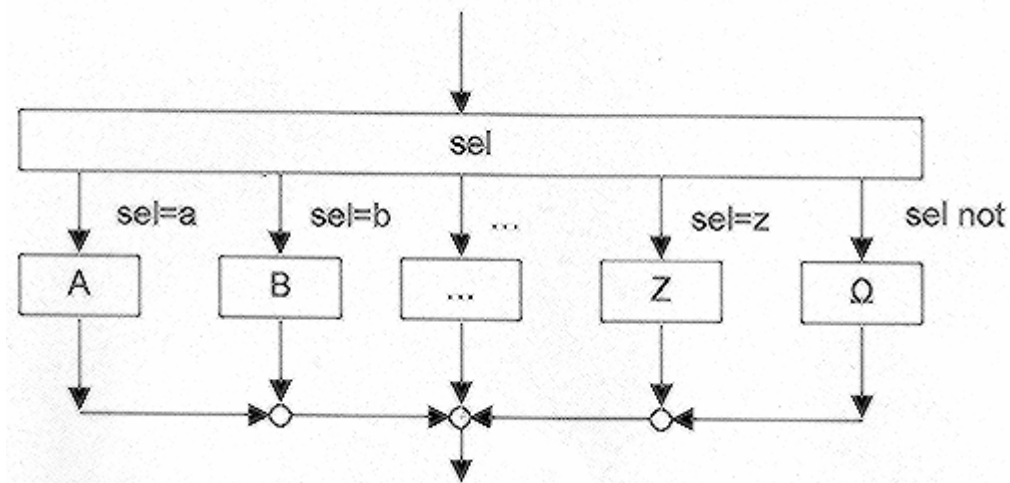
a)

```

CASE sel OF
  a : A; {a□b, ..., a□z}
  b : B; {b□a, ..., b□z}
  .....;
  z : Z      {z□a, z□b, ...}
ELSE Y
END_CASE

```

b)



3.3.2.4 Generalizirana selekcija tipa IF-ORIF-ELSE

a)

```

IF    p1 THEN B1
ORIF  p2 THEN B2
.....
ORIF  pN THEN BN
ELSE  Y
END_IF

```

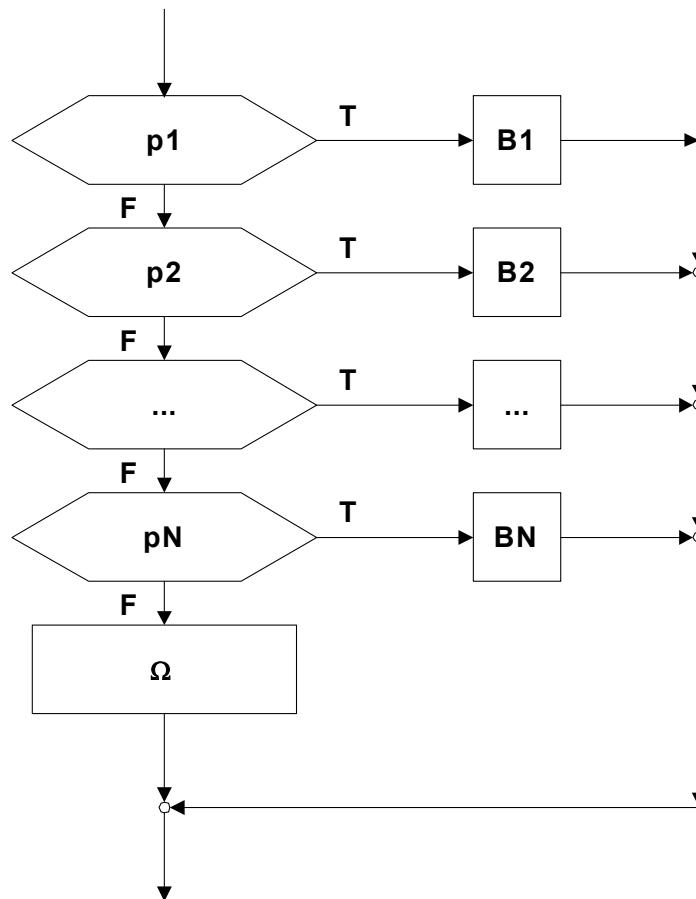
Na primjer, funkcija $y = \text{sgn}(x)$ može se realizirati na način:

```

IF    x < 0 THEN y := -1
ORIF  x = 0 THEN y := 0
ELSE  y := 1
END_IF

```

b)



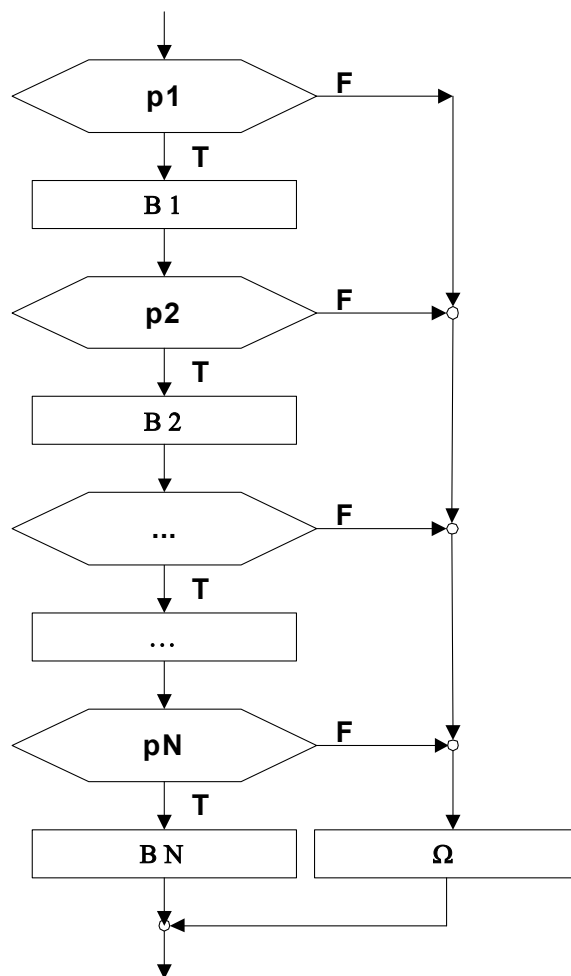
3.3.2.5 Generalizirana selekcija tipa IF-ANDIF-ELSE

a)

```

IF      p1      THEN B1
ANDIF p2      THEN B2
.....
ANDIF pN      THEN BN
ELSE        Y
END_IF
  
```

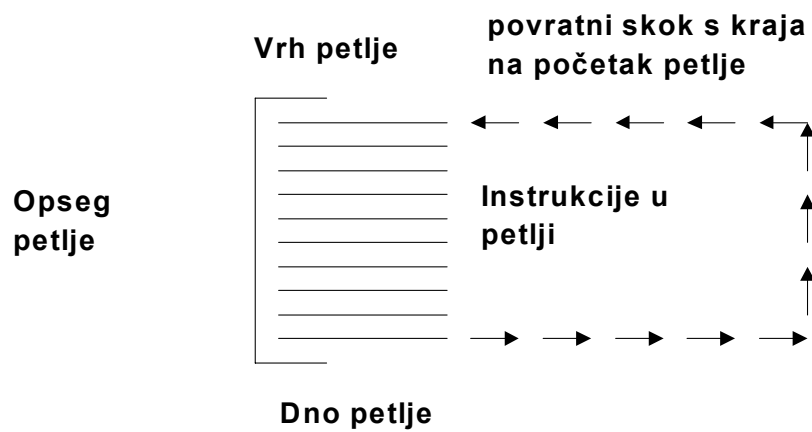
b)



3.3.3 Petlje

Osnovne kategorije iteracija ili petlji su slijedeće:

- petlje sa uvjetnim izlazom na vrhu, dnu i sredini,
- beskonačne petlje sa i bez mehanizma uvjetnog izlaza,
- petlje sa implicitnim brojačem,
- petlje sa eksplicitnim brojačem i
- petlje po elementima skupa.



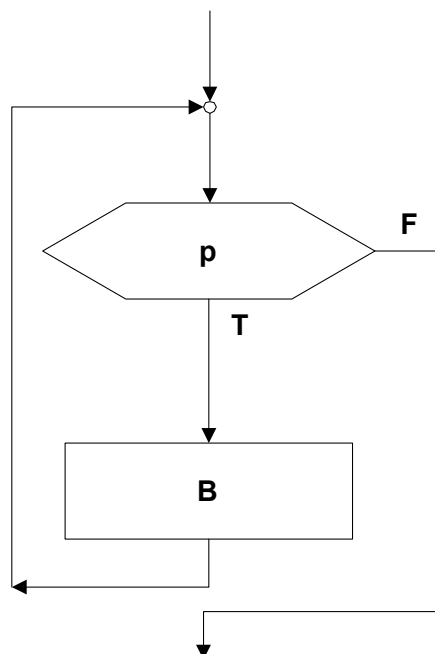
Slika 3.2: Petlje

3.3.3.1 Petlja sa uvjetnim izlazom na vrhu (WHILE-DO)

a)

WHILE p DO B END_WHILE

b)

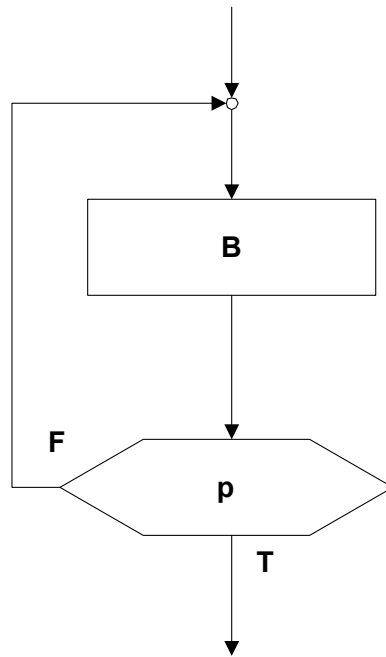


3.3.3.2 Petlja sa uvjetnim izlazom na dnu (REPEAT-UNTIL)

a)

REPEAT B UNTIL p END_REPEAT

b)



3.3.3.3 Petlja sa uvjetnim izlazom na sredini (LOOP-EXIT)

a.1) Notacija u vidu zasebnog konstrukta:

```

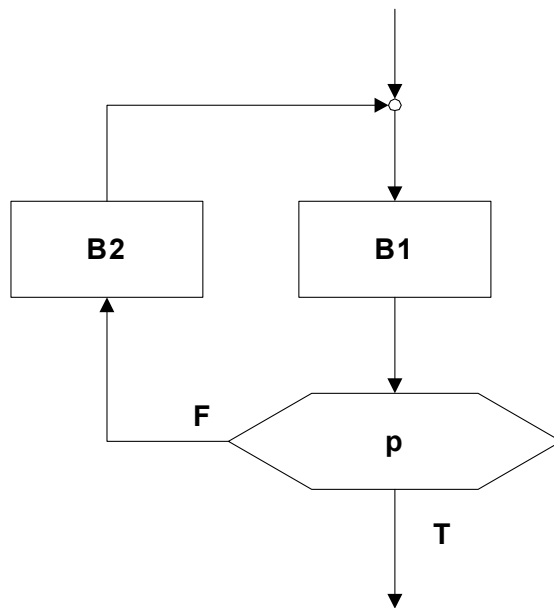
LOOP
    B1
WHEN p EXIT
    B2
END_LOOP
  
```

a.2) Notacija pomoću konstrukata beskonačne LOOP petlje, IF-a i izlaza EXIT:

```

LOOP
    B1 ;
    IF p THEN EXIT END_IF;
    B2
END_LOOP
  
```

b)

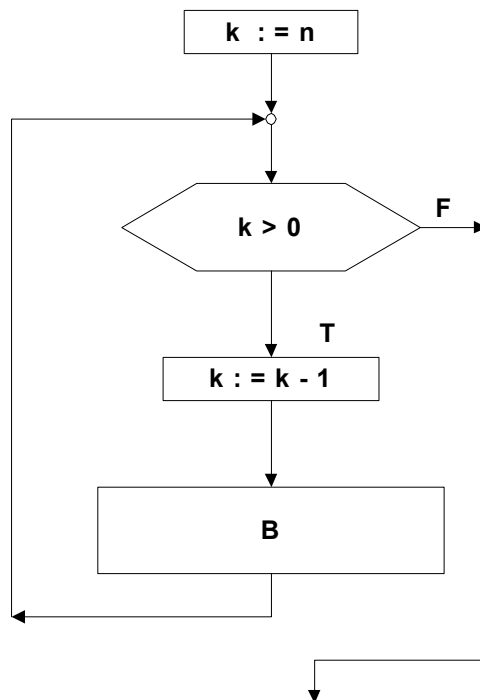


3.3.3.4 Petlja sa implicitnim (internim) brojačem (LOOP-TIMES)

a)

LOOP n TIMES B END_LOOP

b)



3.3.3.5 Petlja sa eksplicitnim (eksternim) brojačem (FOR)

Parametri petlje su slijedeći:

početak = početna vrijednost koju uzima brojač;

Kod ulaza i izlaza numeričkih podataka imamo konverzije koje ilustrira slijedeći primjer:

ULAZ : '1024'

IZLAZ : '1024'

Ulaz teksta i njegovo memoriranje u vidu binarno kodiranog znakovnog niza

0 0 1 1 0 0 0 1 0 0 1 1 0 0 0 0

0 0 1 1 0 0 1 0

0 0 1 1 0 1 0 0

Tekstualno-numerička konverzija i
binarno kodiranog broja konverzija

Numeričko-tekstualna memoriranje u vidu

1024 : 0 1 0 0 0 0 0 0 0 0 0

Ulaznu konverziju u binarno kodirani znakovni niz obavljaju ulazni uređaji (tastatura), a izlaznu konverziju binarno kodiranog znakovnog niza u niz simbola obavljaju izlazni uređaji (pisač ili ekran).

Prilikom prijenosa podataka sa jednog medija na drugi postoje dvije mogućnosti:

- prijenos podataka bez izmjene formata što obuhvaća operacije
čitanje -> prijenos -> upis podataka
- prijenos podataka uz izmjenu formata što obuhvaća operacije
čitanje -> prijenos -> konverzija formata -> prijenos -> upis podataka

U prvom slučaju podaci se čitaju sa ishodišnog medija i bez izmjene doslovno se zapisuju na odredišni medij. Na primjer, podaci se mogu nalaziti u sekvencijalnoj datoteci na disku i ta sekvencijalna datoteka se zatim radi arhiviranja (engl. backup) prenosi na magnetsku vrpcu, bit po bit, bez izmjene njenog binarnog sadržaja.

U drugom slučaju podaci se čitaju sa ishodišnog medija i zatim se obrađuju primjenom programa za konverziju formata. Ovaj program mora sadržavati informaciju o odredišnom formatu podataka kojom se određuje na koji će se način transformirati podaci prije upisa na odredišni medij.

3.4.1 Ulaz i izlaz podataka pomoću ulaznih i izlaznih uređaja računala

Prije pojave interaktivnih terminala dominantan način ulaza podataka u računala bio je pomoću čitača bušenih kartica i papirnatih traka. Ta vrsta ulaznih uređaja više se ne koristi, a unos podataka u računala obavlja se interaktivno pomoću tastature.

Za potrebe učitavanja tekstualnih podataka sa ulazne jedinice (npr. tastature), njihovu konverziju u binarni format i prijenos u operativnu memoriju koristi se naredba READ čiji je opći oblik slijedeći:

READ (ulazna_jedinica, 'format_konverzije') lista_varijabli

```
DEFINE i, j, k : INTEGER ;  
        r, s, t : REAL ;  
        x, y, z : CHARACTER_STRING [1..20] ;
```

```

v      : ARRAY [1..100] OF INTEGER ;
m      : ARRAY [1..100, 1..10] OF REAL ;

```

Naredbom

```

READ i, r, x { ili, što je ekvivalentno, READ (*,*) i, r, x }

```

učitavaju se vrijednosti za i, r, x uz automatsko određivanje formata podataka.

Ako bi se koristila format konverzija onda bi ulazni podaci morali biti definirani u skladu sa format specifikacijom. Na primjer za naredbu

```

READ (*, 'I4, F10.0, A20') i, r, x

```

ulazni podaci mogu biti

```

□123□-987.654□Proizvoljan tekst

```

Prilikom učitavanja elemenata niza moguće je primjeniti FOR petlju na slijedeći način:

```

FOR i := 1 TO 100 DO
    READ v[i]
END_FOR

```

```

READ (v[i], i:=1,100)

```

```

READ ((m[i,j], j:=1,10), i:=1,100) učitavamo komponente matrice po vrstama.

```

Na sličan način, naredbom

```

READ ((m[i,j], i:=1,100), j:=1,10) učitavaju se komponente matrice po kolonama.

```

Za potrebe konverzije memoriranih binarnih podataka u tekstualni format i njihovo ispisivanje na izlaznim uređajima koristi se naredba WRITE čiji je opći obil sličan općem obliku naredbe READ:

```

WRITE (izlazna_jedinica, 'format_konverzije') lista_varijabli

```

Pod izlaznim jedinicama podrazumijevaju se neki od izlaznih uređaja (na primjer ekran ili pisač), datoteka, a također i skalarni ili nizovni tip podataka.

Program za automatsko određivanje formata konverzije (slučaj CARDINAL):

```

DEFINE broj      : CARDINAL ; {broj koji se konvertira}
      ascii      : ARRAY [0..15] OF CHARACTER ; {rezultantni niz
      m          : -1 .. 15 ; {brojač generiranih znakova}
      baza       : CARDINAL ; {baza brojnog sustava konvertiranog
      k          : CARDINAL ; {pomoćna varijabla}
WRITE  'Baza, broj = ' ;
READ  baza, broj ;
m := -1 ;
REPEAT
    m := m + 1 ;
    k := broj / baza ;

```

```

    ascii[m] := char( broj - baza * k + ord('0'));
    broj := k
UNTIL broj = 0
END_REPEAT ;
m := m + 1 ; ascii[m] = ' ';
WRITE (*, '16A1') (ascii[k], k:= m,0,-1

```

Prikazani program izračunava dužinu generiranog broja m koja obuhvaća i jedan vodeći razmak. Ova dužina se zatim koristi da bi se prikazalo m+1 znakova. Konverzija slična prikazanoj, temeljena na decimalnoj bazi, automatski se obavlja kada se realizira naredba

```
WRITE (*,*) broj
```

Ako je "z" znakovni niz, onda se on može iskoristiti kao odredište u koje se upisuje rezultat neke format konverzije. Na primjer, naredbom

```
WRITE (z, '315') i, j, k
```

ispisujemo decimalne znamenke brojeva i, j, k u znakovni niz "z" na isti način kako bi se to činilo koristeći ekran terminala ili papir pisača.

3.4.2 Konverzija podataka tipa A

Konverzija podataka tipa A primjenjuje se za ulaz i izlaz tekstualnih podataka. Pravila prijenosa za ulazno/izlazno polje širine n znakova i memorijsko polje širine m znakova pomoću konverzije tipa An su:

```

Ulaz:   ???X XXXX   XXXX mn      Mem.: mn XXXX   XXXX???
Mem.: mn XXXX      XXXX□□□□      Izlaz:   □□□□XXXXXXX mn

```

Ovdje X označava preneseni znak, ? označava nepreneseni znak, a □ označava razmak. Npr. ako se ulazni niz '12345678' učitava u memorijsku lokaciju koja prima m=4 znaka po format konverzijama A8 i A2 onda će učitani nizovi biti respektivno '5678' i '12□□'. Ako se učitani niz '5678' tiska uz format konverzije A8 i A2 onda će izlazni nizovi biti respektivno '□□□□5678' i '56'.

3.4.3 Konverzija podataka tipa X

Pri učitavanju podataka često se javlja potreba da se određeni podniz podataka preskoči. Na taj način se omogućava da se iz ulaznog niza izdvoje oni podaci koji su korisniku potrebni. Preskakanje podniza dužine k znakova postiže se specifikacijom Xk.

Ulazni niz	Tip konverzije	Vrijednosti učitanih varijabli
ABCDEFGH IJKLMN	A3, X3, A4	ABC, GHIJ
ABCDEFGH IJKLMN	X1,A1,X1,A1,X1,A1,X1,A1,X1,A1	B, D, F, H, J
ABCDEFGH IJKLMN	5(X1, A1)	B, D, F, H, J

3.4.4 Konverzija podataka tipa I

I tip konverzije podataka koristi se za realizaciju tekstualno-numeričke i numeričko-tekstualne konverzije za cjelobrojne podatke. Kod operacije učitavanja podataka ako se navede specifikacija "In" to znači da se iz ulaznog polja uzima podniz dužine "n" znakova počevši od trenutnog položaja pokazivača početka ulaznog podniza. Cjelobrojna veličina koja se nalazi upisana unutar navedenog podniza konvertira se u binarnu numeričku vrijednost.

Ulazni niz	Tip konverzije	Učitani podatak (decimalno)
123456789	I3	123
□□□123□□	I8	123
□□□□□123	I8	123
□□□123□□	I6	123
□□□123□□	I4	1
□□□123□□	I2	0

3.4.5 Konverzija podataka tipa F

F tip konverzije podataka koristi se za realizaciju tekstualno-numeričke i numeričko-tekstualne konverzije isključivo za realne podatke. Kod operacije učitavanja specifikacija "Fn.k" označava da se u ulaznom polju širine "n" znakova nalazi decimalan realan broj koji će se konvertirati u binarnu ekvivalentnu vrijednost.

Ako je broj napisan sa decimalnom točkom, na primjer 123.456, onda se vrši tekstualno-numerička konverzija tog broja bez obzira na vrijednost k, a ako nema decimalne točke onda se podrazumijeva da se decimalna točka automatski umeće k pozicija lijevo od desnog ruba polja.

Kod operacije ispisivanja konverzija Fn.k znači da se binarni realni broj konvertira u tekstualni ekvivalent u polju širine n znakova pri čemu je k znakova desno od decimalne točke. Navedeni tekstualni ekvivalent automatski se oslanja na desni rub polja.

3.4.6 Konverzija podataka tipa E

Uobičajena decimalna notacija realnih brojeva može za vrlo velike i za vrlo male brojeve biti nepraktična jer zahtijeva puno znamenki. Zbog toga se realni brojevi često zapisuju u eksponencijalnoj formi sa normaliziranom decimalnom mantisom i sa decimalnim eksponentom. Broj

$M \times 10^d$, $0.1 \leq M < 1$ (ili $1 \leq M < 10$) zapisuje se kao "M E d".

pri čemu simbol E razdvaja decimalnu mantisu i eksponent. Na primjer,
 $-123.45 = -0.12345 \times 10^3 = -0.12345E+03$

Numeričko-tekstualna konverzija binarnih realnih brojeva u njihove eksponencijalno zapisane decimalne ekvivalente specificira se kodom En.k gdje n označava širinu polja, a k je broj značajnih decimalnih cifara. Na primjer, ako se u memoriji nalazi binarno kodirani broj -123.456789 onda njegova konverzija po specifikaciji E14.5 daje desno oslonjeni rezultat □□-0.12346E+03.

3.5 Klasifikacija osnovnih kontrolnih struktura

Za potrebe vrednovanja i komparacije programskih jezika korisno je cjelovito prikazati klasifikaciju osnovnih kontrolnih struktura:

SEKVENCA..... BEGIN-END

SELEKCIJE

- osnovne
 - grananje IF-THEN-ELSE
 - preskok IF-THEN
 - sa selektorskom CASE
varijablom
- generalizirane
 - sa razgranavanjem
pri neispunjenom uvjetu IF-ORIF-ELSE
 - sa razgranavanjem
pri ispunjenom uvjetu IF-ANDIF-ELSE

ITERACIJE

- sa uvjetnim izlazom
 - na vrhu WHILE-DO
 - na dnu REPEAT-UNTIL
 - na sredini LOOP-EXIT
- bez uvjetnog izlaza (beskonačna petlja)..... LOOP
- sa brojačem
 - implicitnim LOOP-TIMES
 - eksplicitnim
 - aritmetički.....FOR-TO-STEP
FOR-TO-STEP-WHERE
 - sljedbeničkiFOR-TO
FOR-TO-WHERE
 - prethodničkiFOR-DOWNTOW
FOR-DOWNTOW-WHERE
 - skupovni.....FOREACH-IN
FOREACH-IN-WHERE

SKOKOVI

- izlaz iz kontrolne struktureEXIT
- izlaz iz programa.....STOP
- skok sa proizvoljnim odredištemGO TO

3.6 Kontrolne strukture podržane od strane programskih jezika 3GL

KONTROLNE STRUKTURE		PAS	BAS	FOR	COB	C/C++
SEKVENCIJE	BEGIN-END	+	-	-	+	+
SELEKCIJA	IF-THEN-ELSE	+	≈	+	+	+
	IF-THEN-ELSE	+	+	+	+	+
	CASE	+	≈	≈	≈	+
	IF-ORIF-ELSE	-	-	+	+	-
	IF-ANDIF-ELSE	-	-	-	-	-
PETLJE S UVJETOVANIM IZLAZOM	WHILE-DO	+	-	-	+	+
	REPEAT-UNTIL	+	-	-	+	≈
	LOOP-EXIT	-	-	-	-	+
BESKONAČNA PETLJA	LOOP	≈	-	-	≈	≈
	LOOP-TIMES	-	-	-	+	-
PETLJE S BROJAČEM	FOR-TO-STEP	-	+	+	+	+
	FOREACH-IN	-	-	-	-	-
	FOREACH-IN-WHERE	-	-	-	-	-
SKOKOVI	EXIT	-	-	-	-	+
	STOP	-	+	+	+	-
	RETURN (iz potprograma)	+	+	+	+	+
	GO TO	+	+	+	+	+